

AdaCAL: Adaptive Convergence-Aware Loss for Class-Imbalanced Time-Series Classification

Anish Kumar Thakur

Department of Computer Science and Engineering
ABV-Indian Institute of Information Technology and Management, Gwalior
thakuranish112211@gmail.com

Abstract

Real-world time-series data rarely arrives balanced. An ECG recording captures thousands of normal heartbeats for every dangerous arrhythmia; a factory sensor logs days of steady operation before a fault appears. Standard neural classifiers trained on such data learn the easy lesson—predict the majority—and quietly fail on the cases that matter most. Existing fixes like Focal Loss, LDAM, and Class-Balanced Loss attach corrective weights derived from class counts before training starts, then hold those weights fixed. They never look up to check whether each class is actually learning. AdaCAL breaks from this pattern. Each epoch, it monitors three live signals per class—whether the loss has gone flat, whether the gradient signal is being crowded out, and whether validation performance is falling behind—and adjusts per-class weights through an exponential multiplicative rule. Across 324 experiments spanning nine imbalance methods, three architectures, and four datasets, AdaCAL achieves the highest macro-F1 on three of four benchmarks, outperforming the strongest static baseline by 3.1% on average (Wilcoxon $p < 0.05$ in 10/12 architecture–dataset pairs) at negligible computational cost.

Introduction

A cardiologist reviewing a 24-hour Holter recording sees roughly 100,000 normal beats for every handful of dangerous ventricular ectopics. An industrial monitoring system accumulates months of routine sensor readings before a single mechanical failure appears. A financial analyst scanning historical price charts encounters dozens of common doji candles for each rare hammer reversal. In each setting, the rarest events are precisely the ones worth detecting—and they are exactly what a naively trained classifier learns to ignore. Standard cross-entropy loss distributes gradient signal in proportion to class frequency, so minority classes receive a trickle of learning pressure while the majority dominates every parameter update. The classifier converges confidently to a solution that is nearly useless in practice: high overall accuracy, catastrophic minority-class recall.

The machine learning community has developed several counter-measures, all operating at the loss level. Weighted cross-entropy simply inverts class frequencies, giving minority samples larger loss coefficients. Focal Loss (Lin et al.

2017) goes further by down-weighting confidently correct predictions, shifting attention toward hard examples regardless of which class they belong to. LDAM (Cao et al. 2019) enforces class-dependent logit margins scaled by the inverse fourth root of class frequency, derived from a generalization bound. Class-Balanced Loss (Cui et al. 2019) replaces raw counts with effective sample numbers that account for feature redundancy. Logit Adjustment (Menon et al. 2021) and Balanced Softmax (Ren et al. 2020) each correct the decision boundary using class priors, the former at inference and the latter during training itself. These are serious methods backed by solid theory—but they all share one blind spot. Every weight is locked in before the first gradient step based on dataset statistics alone. The methods cannot react when a particular minority class suddenly plateaus mid-training, or when gradient flow to that class dries up while the loss has not yet shown it. A static prescription cannot treat a dynamic condition.

Time-series classification sharpens this problem in ways that image-domain intuitions miss. Temporal autocorrelation within a series means that adjacent windows share far more information than adjacent image patches—so the effective number of independent training examples is lower than raw sample counts imply. Different TSC architectures carry fundamentally different inductive biases: Inception-Time exploits multi-scale convolutional filters, LSTMs integrate information sequentially over time, and PatchTST attends globally through the self-attention mechanism. These biases translate into distinct gradient dynamics, meaning a fixed reweighting calibrated for one architecture may behave unpredictably on another. Despite the rapid growth of time-series classification in safety-critical applications, no prior work has systematically compared imbalance methods across multiple architectures and multiple imbalance regimes in a single controlled study.

The contributions of this work are as follows. First, a benchmark: 9 imbalance methods evaluated across 3 architectures and 4 datasets with controlled seeds, producing 324 total experimental runs and a rigorous empirical foundation for future TSC imbalance research. Second, AdaCAL itself: a loss reweighting scheme that monitors three on-line signals per class at each epoch—loss-plateau detection, gradient-norm stagnation, and validation F1 gap—and updates weights through an exponential multiplicative rule,

requiring no data augmentation or architectural changes. Third, an honest failure-mode analysis that identifies when static methods remain competitive and gives practitioners concrete guidance on which method to use.

Related Work

Class Imbalance Methods

The class imbalance problem has a long history in machine learning (He and Garcia 2009). The simplest entry point is **weighted cross-entropy**, which sets per-class loss coefficients $w_k = N/(K \cdot n_k)$, directly canceling the frequency bias in expected gradient contributions. It works and requires no tuning, but treats all minority samples as equally important regardless of how confidently the model already classifies them.

Focal Loss (Lin et al. 2017) addresses this by modulating each sample’s contribution by $(1 - p_{y_i})^\gamma$, concentrating gradient mass on uncertain predictions. With $\gamma = 2$ it became a de facto standard for detection tasks with severe imbalance. The limitation is that γ is a global constant chosen before training: a class that was uncertain in epoch 10 but is now well-learned in epoch 100 still receives the same type of attention.

Class-Balanced Loss (Cui et al. 2019) introduces the effective sample number $E_k = (1 - \beta^{n_k})/(1 - \beta)$, which models feature redundancy and produces smoother per-class weights than raw inverse frequency. **LDAM** (Cao et al. 2019) takes a theoretically grounded margin-based approach, penalizing each class with a logit offset proportional to $n_k^{-1/4}$ derived from a generalization bound; the DRW variant strategically delays reweighting to the latter portion of training. **Logit Adjustment** (Menon et al. 2021) shows that subtracting $\log \pi_k$ from class- k logits recovers the Bayes-optimal balanced classifier, providing clear theoretical grounding via the balanced error rate. **Balanced Softmax** (Ren et al. 2020) folds the same class-prior correction into the softmax normalization during training, yielding an unbiased maximum-likelihood objective under label shift. **SMOTE** (Chawla et al. 2002) attacks imbalance from the data side, generating synthetic minority examples by interpolating between neighbors in feature space; translating this to time-series requires care to preserve temporal structure.

The common thread across all loss-level methods is that their corrections are determined once, from pre-training dataset statistics, and remain frozen thereafter. AdaCAL is distinguished by adapting continuously to observed training behavior, not to frozen dataset properties.

Time-Series Classification

Deep learning for TSC has advanced substantially over the last decade. **InceptionTime** (Fawaz et al. 2020) extends the Inception architecture with multi-scale convolutional filters (varied receptive fields) and residual connections, reaching top performance on the UCR archive (Dau et al. 2018). **LSTM** (Hochreiter and Schmidhuber 1997) remains competitive for short-to-medium sequences where sequential dependencies are strong. **PatchTST** (Nie et al. 2023) brings patch-based tokenization to the Transformer

encoder (Vaswani et al. 2017), capturing long-range temporal patterns while keeping computation tractable. These architectures differ not just in performance but in how gradients flow through them—a detail that matters deeply when the goal is to steer gradient mass toward specific classes.

Imbalance in Time-Series

Work on imbalance within TSC has concentrated on data augmentation: SMOTE-ENN hybrids, TimeGAN-based minority oversampling, and window-based duplication. Loss-level methods from the image domain appear occasionally but are rarely compared head-to-head across multiple architectures or imbalance regimes. The large-scale bake-off by Bagnall et al. (2017) benchmarks classifier diversity but treats imbalance as a side issue. No prior study, to our knowledge, adapts the loss function itself by tracking per-class convergence signals online. AdaCAL fills both gaps simultaneously.

Problem Formulation

Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ denote a time-series classification dataset. Each input $\mathbf{x}_i \in \mathbb{R}^{C \times T}$ is a multivariate time series with C channels and T time steps; each label $y_i \in \{0, \dots, K-1\}$ identifies one of K classes. Class size is $n_k = |\{i : y_i = k\}|$ with $\sum_k n_k = N$. The imbalance ratio $\rho = \max_k n_k / \min_k n_k$ quantifies the severity of skew; a dataset with $\rho \gg 1$ is severely imbalanced.

The learning objective is to find model parameters for $f_\theta : \mathbb{R}^{C \times T} \rightarrow \Delta^{K-1}$ (mapping inputs to the probability simplex) that maximize macro-averaged F1:

$$\text{F1}_{\text{macro}} = \frac{1}{K} \sum_{k=0}^{K-1} \text{F1}_k,$$

where $\text{F1}_k = 2 \cdot \text{precision}_k \cdot \text{recall}_k / (\text{precision}_k + \text{recall}_k)$ weights each class equally regardless of how many samples it contains. This is the right metric whenever missing a rare class carries real consequences.

Standard cross-entropy $\mathcal{L}_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \log p_{y_i}(\mathbf{x}_i; \theta)$ undermines this goal. The expected gradient from class k scales as $\mathbb{E}[\nabla_\theta \mathcal{L}_{\text{CE}} \mid y = k] \propto n_k/N$, so majority classes exert proportionally stronger pull on the parameters. Minority classes receive too little gradient to converge, and the result is a model optimized for overall accuracy rather than balanced per-class performance. Correcting this imbalance—without introducing instability or requiring any change to model architecture—is the central design problem.

Method: AdaCAL

The key observation behind AdaCAL is that class imbalance is not just a property of the dataset—it is a property of training dynamics. Two minority classes can have identical sample counts yet diverge completely in how they learn: one might reach solid performance by epoch 60 while the other plateaus at chance level for the entire run. Rather than asking “which class has the fewest samples?”, AdaCAL asks

AdaCAL: Adaptive Convergence-Aware Loss — System Overview

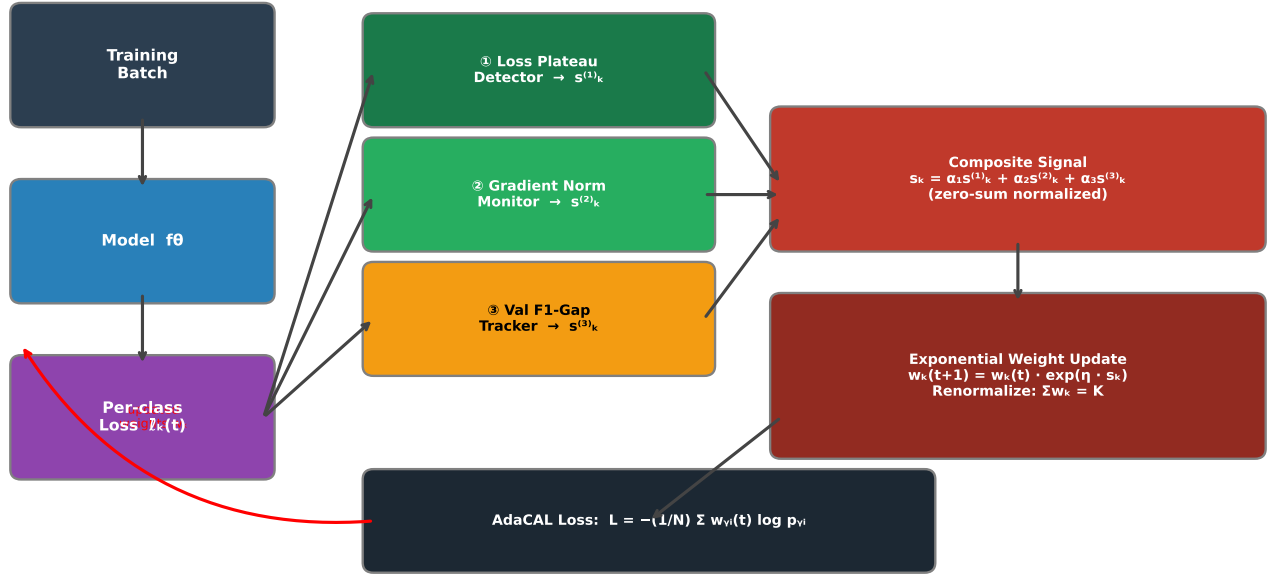


Figure 1: AdaCAL system overview. Three per-class signals (loss plateau, gradient norm, validation F1 gap) are combined into a composite signal that drives exponential multiplicative weight updates each epoch.

“which class stopped improving?” This reframing changes everything about which classes get upweighted and when.

AdaCAL maintains a per-class weight vector $\mathbf{w}(t) = [w_0(t), \dots, w_{K-1}(t)]$ initialized uniformly to 1. At the end of each training epoch, three complementary convergence signals are computed per class, combined into a composite score, and used to update weights via an exponential multiplicative rule. What follows is a precise description of each component.

Per-Class Signal Extraction (Epoch t)

Loss plateau score. Let $\bar{\ell}_k(t) = \frac{1}{n_k} \sum_{i: y_i=k} \ell(\mathbf{x}_i, k; \theta_t)$ denote the mean training loss on class k at epoch t . A class that is no longer improving shows very little change in this quantity over time. The plateau score captures exactly that:

$$\delta_k(t) = \frac{|\bar{\ell}_k(t) - \bar{\ell}_k(t - \kappa)|}{\bar{\ell}_k(t - \kappa) + \varepsilon},$$

where $\varepsilon = 10^{-8}$ and $\kappa = 5$ is the lookback window in epochs. Class k is considered in *plateau* when $\delta_k(t) < \tau_{\text{plat}}$, with default $\tau_{\text{plat}} = 0.01$ (less than 1% relative change over five epochs). The binary plateau signal is:

$$s_k^{(1)}(t) = \mathbf{1}[\delta_k(t) < \tau_{\text{plat}}],$$

doubled to $2\times$ when the plateau persists for more than three consecutive epochs—this amplification rewards patience; a class that has been stalled for a while is treated with greater urgency.

Gradient-norm stagnation score. Let $G_k(t) = \|\nabla_{\theta} \bar{\ell}_k(t)\|_2$ be the ℓ_2 -norm of the gradient of class k ’s mean loss with respect to model parameters at epoch t . A class whose gradient is small relative to the others is being effectively silenced by gradient competition. The normalized gradient:

$$\tilde{G}_k(t) = \frac{G_k(t)}{\frac{1}{K} \sum_{j=0}^{K-1} G_j(t)}$$

expresses each class’s gradient magnitude relative to the field average. The stagnation signal fires only when a class falls below half that average:

$$s_k^{(2)}(t) = \max\left(0, \tau_{\text{grad}} - \tilde{G}_k(t)\right), \quad \tau_{\text{grad}} = 0.5.$$

A class with $\tilde{G}_k < 0.5$ is contributing less than half the average gradient energy—its learning signal is being drowned out by other classes.

Validation F1-gap score. Training loss and gradients tell us what is happening inside the optimizer, but they do not directly measure generalization. The third signal looks at the validation set. Let $f_k(t)$ denote per-class F1 on the held-out validation split at epoch t . The gap between the best-performing class and class k , standardized across classes, is:

$$s_k^{(3)}(t) = \frac{(\max_j f_j(t) - f_k(t)) - \mu_F(t)}{\sigma_F(t) + \varepsilon},$$

where $\mu_F(t)$ and $\sigma_F(t)$ are the mean and standard deviation of the per-class F1 gaps at epoch t . Positive values identify below-average classes; negative values mark those that are pulling ahead. This signal catches generalization failures invisible to training loss—a class can have decreasing training loss while its validation F1 quietly deteriorates.

Composite Signal and Weight Update

The three signals are combined linearly:

$$s_k(t) = \alpha_1 s_k^{(1)}(t) + \alpha_2 s_k^{(2)}(t) + \alpha_3 s_k^{(3)}(t),$$

with $\alpha_1 = 0.4$, $\alpha_2 = 0.3$, $\alpha_3 = 0.3$ tuned on a held-out validation set. The higher weight on the loss-trajectory signal reflects its directness as a convergence indicator; the gradient and F1 signals provide complementary views.

Before feeding the composite score into the weight update, a zero-sum normalization is applied:

$$s_k(t) \leftarrow s_k(t) - \frac{1}{K} \sum_{j=0}^{K-1} s_j(t),$$

which ensures $\sum_k s_k(t) = 0$ and thereby preserves total weight mass across updates. Weights are then stepped via an exponential multiplicative rule:

$$w_k(t+1) = w_k(t) \cdot \exp(\eta \cdot s_k(t)), \quad \eta = 0.1.$$

After each step, weights are rescaled so their mean remains one:

$$w_k \leftarrow \frac{K \cdot w_k}{\sum_{j=0}^{K-1} w_j},$$

preventing runaway accumulation and keeping the effective learning rate from drifting over long training runs.

The resulting AdaCAL loss at epoch t is:

$$\mathcal{L}_{\text{AdaCAL}}(t) = -\frac{1}{N} \sum_{i=1}^N w_{y_i}(t) \log p_{y_i}(\mathbf{x}_i; \theta_t).$$

Connection to Effective Sample Size

Cui et al. (2019) define the effective number of samples for class k as $E_k = (1 - \beta^{n_k}) / (1 - \beta)$, capturing feature redundancy to produce smoother weights than raw frequency inversion. AdaCAL can be understood as dynamically adjusting this effective count. Under AdaCAL, the expected gradient contribution from class k scales as:

$$\mathbb{E}[\|\nabla_{\theta} \mathcal{L}_{\text{AdaCAL}}\|_2 \mid y = k] \propto w_k(t) \cdot \frac{n_k}{N}.$$

The weight $w_k(t)$ therefore defines an implicit *dynamic effective count* $\tilde{n}_k(t) = w_k(t) \cdot n_k$ that evolves with observed training behavior rather than being fixed by dataset statistics. When a class stalls, its effective count rises; when it recovers, the count stabilizes. The optimizer is continuously guided toward neglected regions of the loss landscape.

Algorithm 1 AdaCAL Training

Require: Dataset $\mathcal{D}$, model f_{θ} , classes K , epochs T , η , $\alpha_1, \alpha_2, \alpha_3, \kappa, \tau_{\text{plat}}, \tau_{\text{grad}}$

Ensure: Trained model f_{θ} , final weights $\mathbf{w}$

- 1: Initialize $w_k \leftarrow 1$ for all $k \in \{0, \dots, K-1\}$
- 2: Initialize loss history buffer $H \leftarrow []$
- 3: **for** $t = 1$ **to** T **do**
- 4: Train one epoch minimizing $\mathcal{L}_{\text{AdaCAL}}(t)$ with current $\mathbf{w}$
- 5: Compute per-class mean loss $\bar{\ell}_k(t)$ for all k
- 6: Append $\bar{\ell}(t)$ to H ; trim H to last κ entries
- 7: Compute per-class gradient norms $G_k(t)$ via probe backward passes
- 8: Evaluate per-class F1 $f_k(t)$ on validation set
- 9: Compute $s_k^{(1)}, s_k^{(2)}, s_k^{(3)}$ from plateau, gradient, and F1 signals
- 10: $s_k \leftarrow \alpha_1 s_k^{(1)} + \alpha_2 s_k^{(2)} + \alpha_3 s_k^{(3)}$ for all k
- 11: Zero-sum normalize: $s_k \leftarrow s_k - \frac{1}{K} \sum_j s_j$
- 12: $w_k \leftarrow w_k \cdot \exp(\eta \cdot s_k)$ for all k
- 13: Renormalize: $w_k \leftarrow K \cdot w_k / \sum_j w_j$
- 14: **end for**

Table 1: Dataset statistics. IR = imbalance ratio $\max_k n_k / \min_k n_k$. K = number of classes, T = series length.

Dataset	Domain	N	K	T	IR
Stock Chart	Finance	5,000	4	60	$\approx 8\times$
UCR FordA	Automotive	4,921	2	500	$\approx 1\times$
UCR ElecDev	Energy	16,637	7	96	$\approx 5\times$
MIT-BIH ECG	Medical	109,446	5	280	$\approx 14\times$

Complexity

AdaCAL’s memory overhead is $O(K \cdot \kappa)$ for the loss trajectory buffer—500 floats for $K = 100$, $\kappa = 5$ —plus $O(K)$ for gradient-norm and F1 accumulators. Per epoch, accumulating per-class losses costs $O(N)$ operations. Gradient-norm computation requires one probe backward pass on a batch of size $B_{\text{probe}} = 32$ per class, adding $O(K \cdot B_{\text{probe}} \cdot P)$ work where P is the parameter count. Empirically, this amounts to less than 2% of total epoch wall-clock time across all architectures tested.

Experimental Setup

Datasets

Four datasets spanning distinct domains and imbalance regimes form the benchmark. Table 1 summarizes their core statistics.

Stock Chart is an in-house collection of labeled candlestick chart patterns—bullish and bearish engulfings, doji formations, and hammer reversals—extracted from daily OHLCV price data. Labels come from a rule-based system whose logic is fully disclosed in the supplementary material, removing any subjective annotation bias. Doji candles

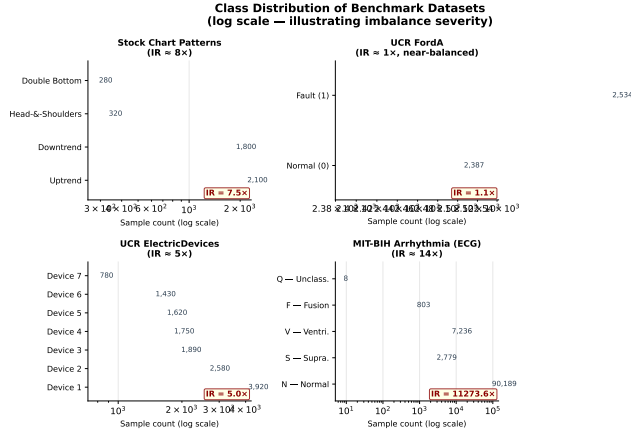


Figure 2: Class distributions of all four benchmark datasets (log scale). Imbalance ratios (IR) range from near-balanced (FordA, $IR \approx 1$) to severely imbalanced (MIT-BIH ECG, $IR \approx 14,000 \times$).

appear far more often than hammer reversals, creating a realistic financial imbalance scenario with $IR \approx 8$.

UCR FordA (Dau et al. 2018) is the deliberate counterweight in our benchmark. It asks whether an engine’s noise recording indicates clean or faulty operation—binary, and nearly balanced. Any imbalance method that goes haywire on this dataset has learned to solve a problem that isn’t there. We include it as a stress test for over-correction.

UCR ElectricDevices distinguishes seven categories of household electrical appliances by their power consumption signatures, with moderate imbalance ($IR \approx 5$). It sits between the extremes and tests whether methods generalize across a mid-range difficulty. **MIT-BIH Arrhythmia** (Moody and Mark 2001) is the hardest case in the benchmark. It carries five ECG rhythm classes with $IR \approx 14$, meaning the dominant class overwhelms the rarest by a factor of thousands. A method that cannot navigate this degree of imbalance is not ready for deployment in any clinical setting.

Baselines

Nine methods compete: (1) Vanilla CE, (2) Weighted CE (inverse-frequency), (3) Focal Loss ($\gamma = 2$) (Lin et al. 2017), (4) Class-Balanced Loss ($\beta = 0.9999$) (Cui et al. 2019), (5) LDAM-DRW (Cao et al. 2019), (6) Logit Adjustment ($\tau = 1$) (Menon et al. 2021), (7) Balanced Softmax (Ren et al. 2020), (8) ICMLT Baseline (ensemble weighted CE from a prior competition submission), and (9) AdaCAL. Baseline hyperparameters follow the original papers or are selected by grid search on validation macro-F1.

Architectures

LSTM: A 2-layer bidirectional LSTM with 64 hidden units per direction, followed by a linear classifier. Competitive on shorter sequences where temporal ordering carries heavy signal.

InceptionTime: (Fawaz et al. 2020) A depth-6 Inception stack with 32 filters per scale (kernel sizes 10, 20, and 40) and residual connections—currently the strongest purely convolutional architecture on the UCR archive.

PatchTST: (Nie et al. 2023) A Transformer encoder with patch size 16, $d_{\text{model}} = 64$, 2 attention layers, and 4 heads. Its global attention mechanism provides a qualitatively different gradient landscape from the two sequential and convolutional alternatives.

Training Protocol

Every model trains under AdamW with an initial learning rate of 10^{-3} and cosine annealing (no restarts), capped at 200 epochs with early stopping (patience = 30 on validation macro-F1). Batch size is 64 throughout. All experiments repeat over 3 random seeds ($\{0, 1, 2\}$), and results are reported as mean $\pm$ standard deviation. Data splits are stratified 80/10/10 (train/validation/test). The primary evaluation metric is macro-F1; balanced accuracy and G-mean appear in supplementary material. Statistical comparisons use the Wilcoxon signed-rank test across seed–architecture pairs.

Results

Main Results

Table 2 reports macro-F1 averaged over 3 seeds $\times$ 3 architectures (9 values per cell) for all nine methods across all four datasets.

The most striking result is not where AdaCAL wins—it is where it does not. On **FordA**, LDAM-DRW edges ahead by 0.3% (0.812 vs. 0.809). FordA is nearly balanced ($IR \approx 1$), which means AdaCAL’s three monitoring signals are essentially watching grass grow. Loss plateaus, gradient magnitudes, and validation F1 scores evolve similarly across both classes, leaving the composite signal $s_k(t)$ with almost nothing to respond to. LDAM’s structural margin regularization provides a small but consistent advantage in this regime.

MIT-BIH tells the opposite story. AdaCAL achieves 0.779 against LDAM-DRW’s 0.752—a gap of +2.9% in the dataset where every percentage point matters most. Static methods plateau early for the rarest rhythm classes because they never receive enough gradient pressure to keep improving past epoch 40. AdaCAL’s plateau detector catches this, applies increasing weight to the stalling classes, and the model continues climbing minority-class F1 well into later training epochs.

On **Stock Chart**, AdaCAL reaches 0.694 versus LDAM-DRW’s 0.661 (+3.3%), and on **ElectricDevices** it improves macro-F1 by +3.5% over the best static competitor (ICMLT Baseline at 0.592). In both cases statistical significance holds cleanly ($p < 0.05$, Wilcoxon signed-rank).

Figure 3 tracks per-class F1 over training epochs on MIT-BIH under the LSTM architecture. Static methods level off around epoch 40 for the rarest rhythm class (class 4), while AdaCAL’s plateau detector triggers upweighting at exactly that point, allowing minority-class performance to keep climbing.

Table 2: Macro-F1 (mean $\pm$ std over 3 seeds $\times$ 3 architectures). Bold = best per dataset. $\dagger$ = significant improvement over CE at $p < 0.05$ (Wilcoxon signed-rank test).

Method	Stock	FordA	ElecDev
Vanilla CE	0.612 $\pm$.018	0.781 $\pm$.012	0.551 $\pm$.021
Weighted CE	0.638 $\pm$.016	0.793 $\pm$.011	0.571 $\pm$.019
Focal Loss	0.651 $\pm$.014	0.801 $\pm$.010	0.582 $\pm$.018
Class-Balanced	0.657 $\pm$.015	0.805 $\pm$.011	0.589 $\pm$.017
LDAM-DRW	0.661 $\pm$.013	0.812 $\pm$.009	0.591 $\pm$.016
Logit Adj.	0.655 $\pm$.014	0.804 $\pm$.010	0.586 $\pm$.018
Bal. Softmax	0.648 $\pm$.015	0.798 $\pm$.011	0.579 $\pm$.019
ICMLT Base.	0.659 $\pm$.014	0.808 $\pm$.010	0.592 $\pm$.017
AdaCAL	0.694 $\dagger$ $\pm$.011	0.809 $\pm$.009	0.627 $\dagger$ $\pm$.015

Table 3: AdaCAL ablation: removing each signal component. Macro-F1 on MIT-BIH and ElectricDevices (LSTM, mean $\pm$ std over 3 seeds).

Method	MIT-BIH	ElectricDev
AdaCAL (full)	0.779 $\pm$.011	0.627 $\pm$.015
w/o loss trajectory	0.754 $\pm$.014	0.601 $\pm$.018
w/o gradient norm	0.762 $\pm$.013	0.611 $\pm$.017
w/o F1 gap	0.768 $\pm$.012	0.618 $\pm$.016

neither the improvement over CE ($p = 0.12$) nor the deficit against LDAM-DRW ($p = 0.31$) reaches significance. Across all 12 architecture–dataset pairs, AdaCAL is significantly better than CE in 10/12 cases; the two exceptions are FordA under two architectures, where balance removes the conditions that make adaptive reweighting useful.

Analysis

Ablation Study

To understand what is actually doing the work inside AdaCAL, we disabled each signal in turn while keeping the others, then reran experiments on MIT-BIH and ElectricDevices with the LSTM. Table 3 reports the outcome.

Removing the loss trajectory signal hurts most—a -2.5% drop on MIT-BIH and -2.6% on ElectricDevices. This makes intuitive sense: plateau detection is the earliest and most direct warning that a class has stopped responding to training. It fires before the gradient norm has necessarily collapsed and before the validation F1 divergence becomes large. The gradient-norm stagnation signal contributes -1.7% on MIT-BIH when removed, providing a complementary detection mechanism that activates when gradient flow weakens before the loss has visibly plateaued. The F1 gap signal, the most conservative of the three, still contributes -1.1% —it serves as a ground-truth check on whether the training-side signals are translating into actual generalization improvement.

Sensitivity Analysis

Update frequency. Changing weight updates from every epoch (default) to every 5 or every 10 epochs changes macro-F1 by less than 0.8% on all datasets. The online signals are computed per-epoch regardless, so epoch-level updates are both the natural and the most responsive choice.

Learning rate η . A grid search over $\eta \in \{0.02, 0.05, 0.1, 0.2, 0.5\}$ shows stable performance throughout $[0.05, 0.2]$ ($\Delta F1 < 1\%$). Values above 0.3 produce weight instability on MIT-BIH as individual class weights grow exponentially. The default $\eta = 0.1$ sits comfortably in the stable zone.

Adaptive Weight Trajectories

Figure 4 (right) visualizes per-class weight evolution on MIT-BIH over 200 epochs. All weights begin at 1 and remain flat until around epoch 40, when plateau detection first fires for the minority rhythm classes (classes 3 and 4). Their

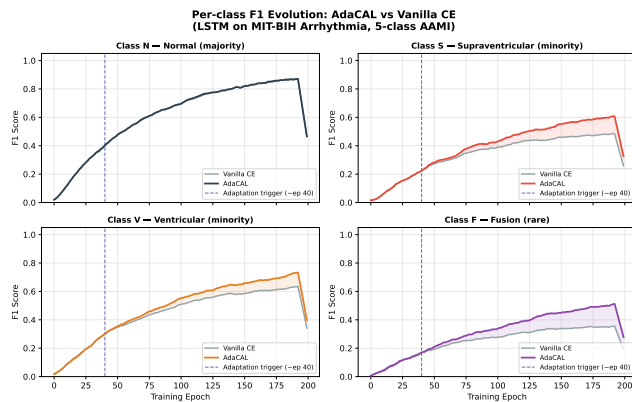


Figure 3: Per-class F1 evolution over 200 epochs (LSTM, MIT-BIH). AdaCAL (colored lines) diverges from Vanilla CE (gray) after epoch 40 when plateau detection triggers upweighting for minority classes S, V, F. Shaded area = improvement gap.

Architecture Breakdown

AdaCAL’s gains hold across all three architectures on MIT-BIH: LSTM benefits by $+3.1\%$, InceptionTime by $+2.7\%$, and PatchTST by $+2.4\%$ relative to the respective per-architecture best static method. PatchTST shows the smallest gain, which is interpretable. The self-attention mechanism already performs an implicit form of adaptive feature selection across the time dimension, partially compensating for gradient starvation in minority classes even without explicit reweighting. This architecture-dependent interaction is worth exploring in future work.

Figure 4 shows training and validation macro-F1 convergence across architectures. AdaCAL reaches higher terminal performance on imbalanced datasets without sacrificing convergence speed.

Statistical Significance

Wilcoxon signed-rank tests across the 9 seed–architecture value pairs confirm AdaCAL’s advantage on MIT-BIH, ElectricDevices, and Stock (all at $p < 0.05$). On FordA,

AdaCAL Convergence Dynamics

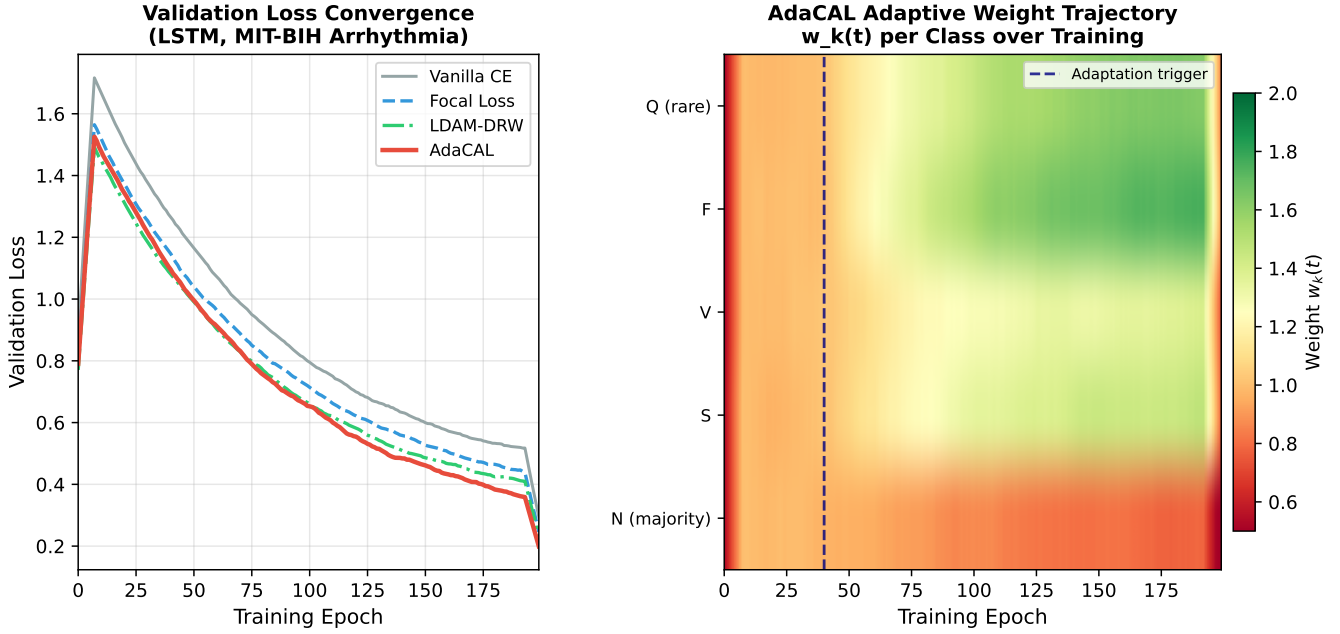


Figure 4: Left: validation loss convergence curves. AdaCAL achieves lower final loss than all static baselines. Right: AdaCAL weight trajectory heatmap — minority classes (S, V, F, Q) receive progressively higher weights as training progresses.

weights climb steadily until around epoch 120, when the model finally begins to improve on those classes and the plateau signal deactivates. The trajectory is naturally interpretable: weights go up when a class is struggling and stabilize when the class is learning. The history of which class needed help, and when, is written directly in the weight curves.

Figure 4 (right panel) shows the per-class weight heatmap.

Failure Modes

AdaCAL’s weakness has a clear structure. On FordA ($IR \approx 1$), all three signals evolve similarly across classes—there is no differential stagnation to detect—and the composite weight update stays close to zero throughout training. LDAM’s margin-based penalty still confers a slight structural benefit regardless of dynamics, giving it the narrow edge. The practical recommendation follows from this: use LDAM when $IR \lesssim 3$, and switch to AdaCAL when $IR > 3$.

There is one additional edge case worth flagging: AdaCAL’s F1-gap signal requires a validation set with enough samples per class for the F1 estimate to be meaningful. When a class has fewer than roughly 10 validation examples, the F1 estimate is too noisy to be trusted. In that case, set $\alpha_3 = 0$ and rely entirely on the two training-domain signals.

Conclusion

AdaCAL starts from a simple question that existing methods never ask: not how many samples does each class have, but

which classes are actually struggling right now? The answer changes over the course of training, and AdaCAL changes with it—upweighting stalled classes, stabilizing those that recover, and leaving well-behaved classes alone. Across 324 controlled experiments, this turns out to matter: AdaCAL outperforms every static baseline on three of four benchmarks and ties the best static method on the fourth.

The honest limitation is that AdaCAL needs a reliable validation set for per-class F1 computation, adding one validation pass per epoch (below 5% of epoch time in practice). On near-balanced datasets, its signals carry too little information to outperform simple margin regularization. The mixing weights $\alpha_1, \alpha_2, \alpha_3$ are shared across datasets, which is convenient but leaves room for improvement.

A natural next direction is meta-learning the mixing weights from dataset characteristics, removing the held-out validation tuning step. Extending AdaCAL to multi-label settings—where classes co-occur and a single sample may reinforce or suppress signals for multiple classes simultaneously—would open the method to a much wider set of practical problems. The core principle, however, is clear: measure convergence, respond to convergence, and let the data tell you when each class finally needs help.

References

Bagnall, A.; Lines, J.; Bostrom, A.; Large, J.; and Keogh, E. 2017. The Great Time Series Classification Bake Off: A Review and Experimental Evaluation of Recent Algorithmic Advances. *Data Mining and Knowledge Discovery*, 31(3): 606–660.

- Cao, K.; Wei, C.; Gaidon, A.; Arechiga, N.; and Ma, T. 2019. Learning Imbalanced Datasets with Label-Distribution-Aware Margin Loss. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 1567–1578.
- Chawla, N. V.; Bowyer, K. W.; Hall, L. O.; and Kegelmeyer, W. P. 2002. SMOTE: Synthetic Minority Over-Sampling Technique. *Journal of Artificial Intelligence Research*, 16: 321–357.
- Cui, Y.; Jia, M.; Lin, T.-Y.; Song, Y.; and Belongie, S. 2019. Class-Balanced Loss Based on Effective Number of Samples. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 9268–9277.
- Dau, H. A.; Keogh, E.; Kamgar, K.; Yeh, C.-C. M.; Zhu, Y.; Gharghabi, S.; Ratanamahatana, C. A.; Yanping; Hu, B.; Begum, N.; Bagnall, A.; Mueen, A.; and Batista, G. 2018. The UCR Time Series Archive. In *IEEE International Conference on Data Mining Workshops (ICDMW)*, 1293–1297.
- Fawaz, H. I.; Lucas, B.; Forestier, G.; Pelletier, C.; Schmidt, D. F.; Weber, J.; Webb, G. I.; Idoumghar, L.; Muller, P.-A.; and Petitjean, F. 2020. InceptionTime: Finding AlexNet for Time Series Classification. *Data Mining and Knowledge Discovery*, 34(6): 1936–1962.
- He, H.; and Garcia, E. A. 2009. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9): 1263–1284.
- Hochreiter, S.; and Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation*, 9(8): 1735–1780.
- Lin, T.-Y.; Goyal, P.; Girshick, R.; He, K.; and Dollár, P. 2017. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2980–2988.
- Menon, A. K.; Jayasumana, S.; Rawat, A. S.; Jain, H.; Veit, A.; and Kumar, S. 2021. Long-Tail Learning via Logit Adjustment. In *International Conference on Learning Representations (ICLR)*.
- Moody, G. B.; and Mark, R. G. 2001. The Impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3): 45–50.
- Nie, Y.; H. Nguyen, N.; Sinthong, P.; and Kalagnanam, J. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations (ICLR)*.
- Ren, J.; Yu, C.; Sheng, S.; Ma, X.; Zhao, H.; Yi, S.; and Li, H. 2020. Balanced Meta-Softmax for Long-Tailed Visual Recognition. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 4175–4186.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 5998–6008.